



Testes com IA (Unitários, E2E e API)

Testes Unitários Assistidos por IA

MÓDULO 1 SEMANA 06 - DIA 01



asCTI.SC

AGENDA

- Confiabilidade de código assistido por IA
- Fundamentos de testes unitários
- Estrutura Arrange / Act / Assert
- Geração automática de testes com IA
- Casos positivos, negativos e edge cases
- Demonstração prática com Pytest
- Desafio prático guiado

OBJETIVO

Ao final da aula você será capaz de:

- Entender a função dos testes unitários
- Estruturar testes usando AAA
- Gerar testes automaticamente com IA
- Identificar casos de borda e cenários críticos
- Validar a qualidade de testes gerados por IA

CRONOGRAMA

Semana 6

- **Segunda (11/05):** Testes unitários assistidos por IA
- **Terça (12/05):** Testes E2E e testes de API com IA
- **Quinta (14/05):** Cobertura, análise de riscos e apresentação dos mini projetos

Por que testar código?

```
def calcular_frete(valor_compra, distancia_km):  
    if valor_compra >= 200:  
        return 0  
    if distancia_km <= 10:  
        return 10  
    return 20
```

Exemplo:

Compra: R\$ 250

Distância: 50 km

Resultado esperado:

Frete grátis

Mas uma alteração incorreta pode retornar:

Frete = R\$ 20

Como garantir que essas regras continuarão funcionando depois de uma alteração?

Transformando regras em testes

Compra acima de R\$ 200 → Frete grátis

```
def test_frete_gratis():  
    resultado = calcular_frete(250, 50)  
  
    assert resultado == 0
```

Distância até 10 km → Frete R\$ 10

```
def test_frete_10_reais():  
    resultado = calcular_frete(100, 5)  
  
    assert resultado == 10
```

Distância até 10 km → Frete R\$ 20

```
def test_frete_20_reais():  
    resultado = calcular_frete(100, 20)  
  
    assert resultado == 20
```

Regra de negócio

Teste

Validação
automática

Quando uma alteração quebra uma regra

A regra de frete grátis nunca será executada

```
def calcular_frete(valor_compra, distancia_km):
```

```
    if distancia_km <= 10:  
        return 10
```

```
    if distancia_km > 10:  
        return 20
```

```
    if valor_compra >= 200:  
        return 0
```

```
===== FAILURES =====
```

```
FAILED test_frete_gratis
```

```
Expected: 0
```

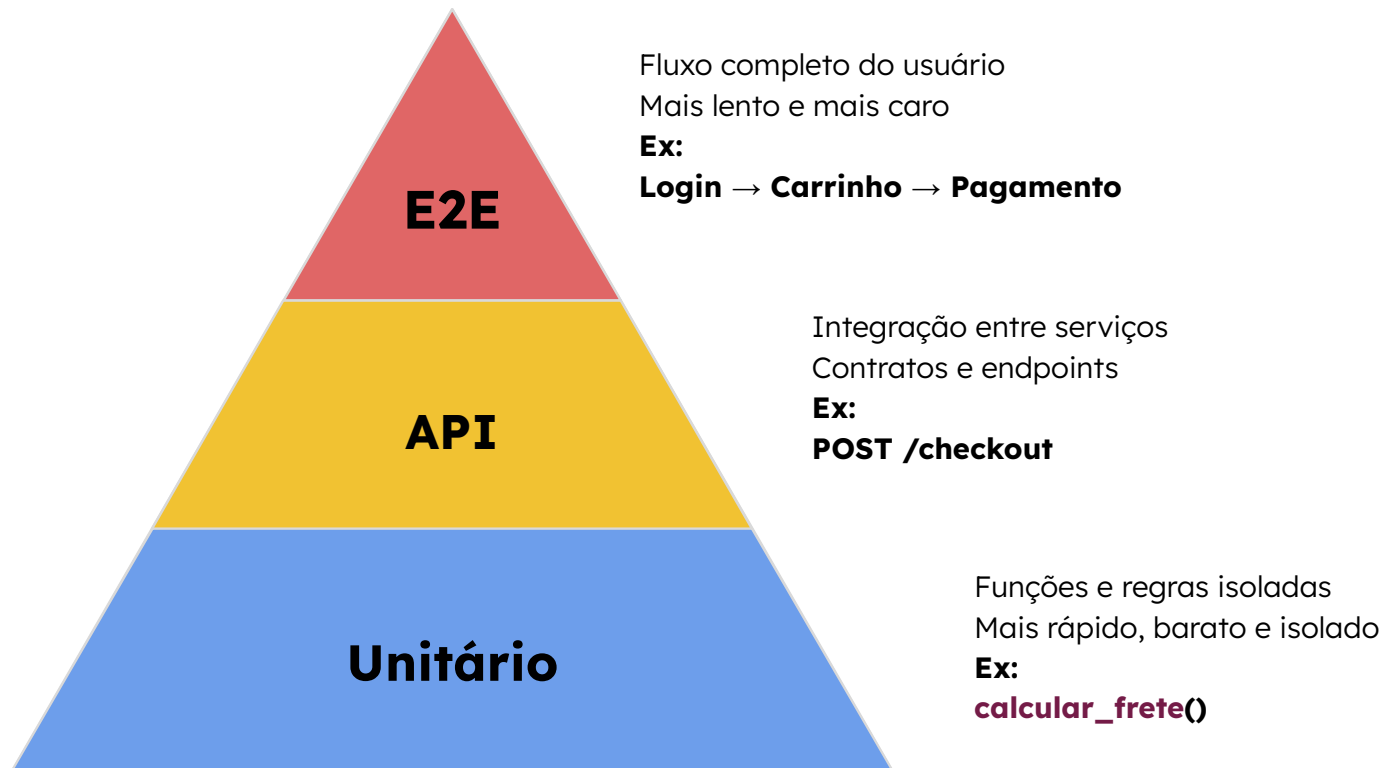
```
Received: 20
```

O sistema continua funcionando, mas a regra de negócio foi quebrada.

Diferentes níveis de validação automatizada

Tipo de teste	O que valida	Exemplo
Unitário	Função ou regra isolada	<code>calcular_frete()</code>
Integração	Comunicação entre componentes	API + Banco
API	Contrato dos endpoints	<code>POST /checkout</code>
E2E	Fluxo completo do usuário	Compra no app

Diferentes níveis de validação automatizada

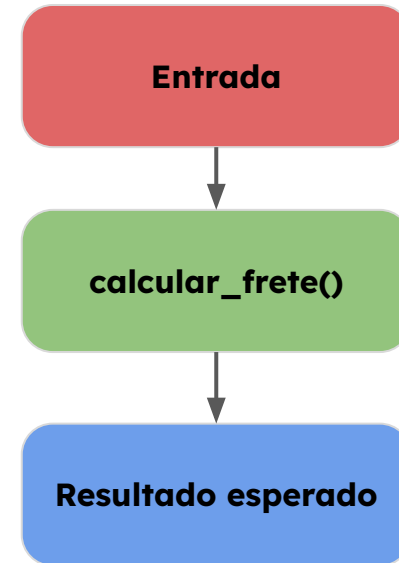


O que é teste unitário?

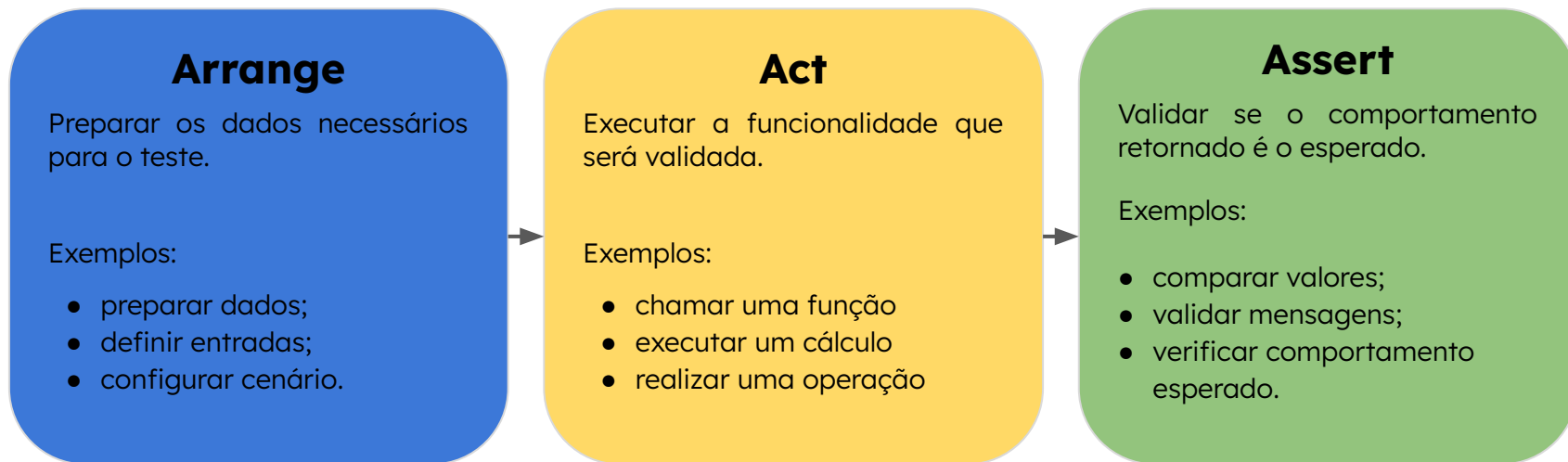
Teste unitário valida uma pequena unidade de comportamento do sistema de forma isolada.

Escopos comuns de validação:

- funções
- métodos
- regras de negócio
- cálculos
- validações
- transformações de dados



Estrutura de um teste unitário



AAA ajuda a organizar testes de forma clara e previsível.

Arrange, Act e Assert na prática

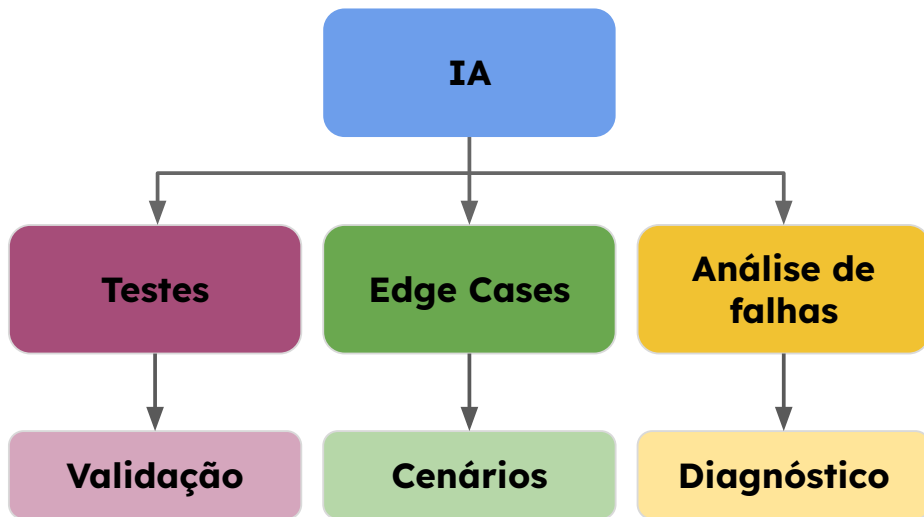
```
def test_frete_gratis():  
  
    # Arrange (Preparação dos dados)  
    valor_compra = 250  
    distancia_km = 50  
  
    # Act (Execução da função)  
    resultado = calcular_frete(valor_compra, distancia_km)  
  
    # Assert (Validação do resultado)  
    assert resultado == 0
```

O que deve ser validado em um teste?

Exemplos de cenários

Tipo de cenário	Entrada	Resultado esperado
Caminho feliz	Compra = 250, distância = 50	Frete grátis
Distância curta	Compra = 80, distância = 3	Frete R\$ 10
Distância longa	Compra = 120, distância = 25	Frete R\$ 20
Limite da promoção	Compra = 200, distância = 15	Frete grátis
Valor zerado	Compra = 0, distância = 8	Frete R\$ 10
Entrada inválida	Compra = -50, distância = -2	Erro ou validação
Tipo inválido	Compra = "100", distância = 5	Erro ou validação

IA no ciclo de testes automatizados



Validação (IA pode gerar testes)

```
assert calcular_frete(250, 50) == 0
```

Cenários (IA pode sugerir casos importantes)

Compra = 200

Distância = 10

Diagnóstico (IA pode explicar erros)

Expected: 0

Received: 20

Engenharia de prompts para geração de testes

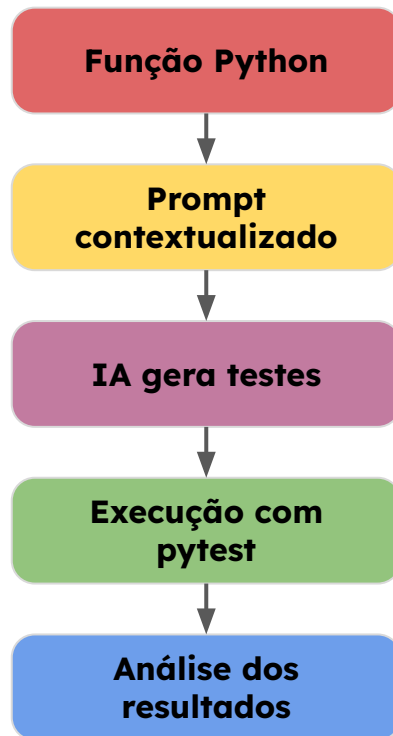
Prompt genérico	Prompt contextualizado
Crie testes para essa função. Problemas • pouca contextualização • regras não especificadas • cenários incompletos • testes genéricos	Gere testes unitários em Python usando pytest para a função <code>calcular_frete()</code>. Considere: - cenários positivos - entradas inválidas - edge cases - limite de compra em R\$ 200 - limite de distância em 10 km

Prompts melhores produzem testes mais úteis e mais relevantes.

Demonstração prática: geração de testes com IA

Fluxo da validação assistida por IA

- Analisar uma função Python
- Gerar testes unitários
- Sugerir edge cases
- Identificar possíveis falhas
- Acelerar a validação automatizada



Função Python a ser testada

```
def calcular_frete(valor_compra, distancia_km):  
  
    if valor_compra < 0 or distancia_km < 0:  
        raise ValueError("Valores inválidos")  
  
    if valor_compra >= 200:  
        return 0  
  
    if distancia_km <= 10:  
        return 10  
  
    return 20
```

Regra	Resultado
Compra $\geq$ R\$ 200	Frete grátis
Distância $\leq$ 10 km	Frete R\$ 10
Distância $>$ 10 km	Frete R\$ 20
Valores negativos	ValueError

Antes de gerar testes, é necessário entender o comportamento esperado da função.

Engenharia de prompts para geração de testes

Parte do prompt	Objetivo
<code>função calcular_frete</code>	Definir função analisada
<code>arquivo frete.py</code>	Utilizar contexto do projeto
<code>testes unitários</code>	Definir o tipo de validação
<code>Python usando pytest</code>	Definir linguagem e framework
<code>cenários positivos</code>	Garantir comportamento esperado
<code>entradas inválidas</code>	Validar tratamento de erro
<code>edge cases</code>	Validar limites críticos
<code>limite de compra R\$ 200</code>	Explicitar regra de negócio
<code>Nomes descritivos</code>	Melhorar legibilidade e manutenção

Prompt

Analise a `função calcular_frete` no `arquivo frete.py`.

Gere `testes unitários` em Python usando `pytest`.

Considere:

- `cenários positivos`;
- `entradas inválidas`;
- `edge cases`;
- `limite de compra em R$ 200`;
- `limite de distância em 10 km`.

Inclua `nomes claros` para os testes.

Testes gerados pela IA

```
import pytest
from frete import calcular_frete

def test_frete_gratis():
    assert calcular_frete(250, 50) == 0

def test_frete_10_reais():
    assert calcular_frete(100, 5) == 10

def test_frete_20_reais():
    assert calcular_frete(100, 20) == 20

def test_valor_negativo():
    with pytest.raises(ValueError):
        calcular_frete(-10, 5)
```

Comportamentos identificados pela IA

Comportamento	Tipo de teste
Frete grátis	Cenário positivo
Distância $\leq$ 10 km	Regra de negócio
Compra < 200 e distância > 10 km	Cenário alternativo
Valores negativos	Cenário inválido

A IA inferiu cenários a partir das regras implementadas no código.

Interpretando falhas nos testes

Exemplo de falha identificada pelo pytest

```
test_frete.py FF.....F..FFF
```

```
===== FAILURES =====  
----- test_frete_gratis_quando_valor_igual_a_200 -----
```

```
> def test_frete_gratis_quando_valor_igual_a_200():  
    assert calcular_frete(200, 50) == 0  
E     assert 20 == 0  
E     + where 20 = calcular_frete(200, 50)
```

```
test_frete.py:6: AssertionError
```

Informação	Interpretação
AssertionError	O resultado não corresponde ao esperado
assert 20 == 0	O teste esperava frete grátis
where 20 = calcular_frete(200, 50)	A função retornou 20
test_frete_gratis_quando_valor_igual_a_200	Regra quebrada
FF....F..FFF	Parte dos testes falhou

Identificando a regressão

```
def calcular_frete(valor_compra, distancia_km):
```

```
    if valor_compra < 0 or distancia_km < 0:  
        raise ValueError("Valores inválidos")
```

```
    if distancia_km <= 10:  
        return 10
```

```
    if distancia_km > 10:  
        return 20
```

```
    if valor_compra >= 200:  
        return 0
```

Problema	Impacto
Validação de distância executa primeiro	Frete grátis nunca é alcançado
return 10 e return 20 encerram a função	A validação de R\$ 200 é ignorada
Ordem das regras foi alterada	Regressão introduzida

Os testes identificaram que a alteração modificou o comportamento esperado da regra de negócio.

Corrigindo a implementação

```
def calcular_frete(valor_compra, distancia_km):
```

```
    if valor_compra < 0 or distancia_km < 0:  
        raise ValueError("Valores inválidos")
```

```
    if valor_compra >= 200:  
        return 0
```

```
    if distancia_km <= 10:  
        return 10
```

```
    return 20
```

Ajuste realizado	Resultado
Regra de frete grátis voltou para o topo	Prioridade restaurada
Fluxo lógico reorganizado	Regressão removida
return antecipado evitado	Regra de negócio preservada

Exercício prático: geração de testes com IA

Durante a prática, todos deverão:

- Analisar funções Python;
- Gerar testes unitários com apoio de IA;
- Executar os testes com **pytest**;
- Interpretar falhas;
- Validar os cenários gerados.

Etapa	Objetivo
Desafio 1	Validação de entrada
Desafio 2	Regras e prioridades
Desafio 3	Fluxos e prioridades condicionais

Entrega esperada

- arquivos **test_*.py**;
- testes executados com **pytest**;
- cenários positivos, negativos e limites (edge cases);
- interpretação dos resultados.

Desafio 1: validação de senha

```
def validar_senha(senha):
```

```
    if len(senha) < 8:  
        return False
```

```
    if senha.islower():  
        return False
```

```
    if senha.isalpha():  
        return False
```

```
    return True
```

Objetivo da atividade

Gerar testes unitários usando pytest para validar diferentes tipos de senha.

Regras de negócio

- Senha com menos de 8 caracteres = **Inválida**
- Senha sem letra maiúscula = **Inválida**
- Senha sem número = **Inválida**
- Senha com 8+ caracteres, pelo menos 1 maiúscula e pelo menos 1 número = **Válida**

1. Quais regras tornam a senha inválida?
2. Quais cenários precisam ser protegidos pelos testes?

Desafio 2: cálculo de desconto

```
def calcular_desconto(valor_compra, cliente_vip):
```

```
    if valor_compra < 0:
        raise ValueError("Valor inválido")
```

```
    if cliente_vip:
        return valor_compra * 0.85
```

```
    if valor_compra >= 300:
        return valor_compra * 0.90
```

```
    return valor_compra
```

1. Quais regras definem os descontos aplicados?
2. Existe prioridade entre as regras?
3. Quais cenários devem ser validados pelos testes?

Objetivo da atividade

Validar regras de desconto e prioridade de execução através de testes automatizados.

Regras de negócio

- Valor da compra menor que 0 = **ValueError**
- Cliente VIP = **15% de desconto**
- Compra maior ou igual a R\$ 300 = **10% de desconto**
- Demais casos = **Sem desconto**
- Cliente VIP com compra $\geq$ R\$ 300 = **Aplica desconto VIP de 15%**

Desafio 3: classificação de chamados

```
def classificar_chamado(tipo_problema, cliente_premium):
```

```
    if tipo_problema == "sem_sinal":  
        return "alta"
```

```
    if tipo_problema == "cobranca" and cliente_premium:  
        return "alta"
```

```
    if tipo_problema == "cobranca":  
        return "media"
```

```
    return "baixa"
```

Objetivo da atividade

Validar fluxos condicionais e regras de prioridade na classificação de chamados.

Regras de negócio

- Sem sinal = prioridade **alta**
- Cobranca + cliente premium = prioridade **alta**
- cobrança + cliente comum = prioridade **média**
- outros casos = prioridade **baixa**

O código implementa corretamente as regras esperadas?

Erros comuns durante os testes automatizados

Erro comum	Consequência
Validar comportamento incorreto	Testes aprovam bugs
Não revisar o código gerado pela IA	Falhas passam despercebidas
Ignorar edge cases	Cobertura incompleta
Criar testes genéricos	Baixa proteção contra regressões
Não interpretar falhas do pytest	Debugging superficial
Testar implementação em vez da regra	Fragilidade dos testes

Testes automatizados devem validar regras esperadas, não apenas repetir o comportamento atual do código.

Correção guiada dos desafios

Pontos que deveriam ter sido identificados

Desafio	Ponto crítico
Senha	Senha apenas numérica
Desconto	Prioridade VIP
Chamado	Ordem das regras

1. Os testes realmente protegiam as regras esperadas?
2. A IA identificou corretamente todos os cenários?
3. Algum bug passou despercebido?

Principais aprendizados do dia

Conteúdos abordados nesta aula:

- Fundamentos de testes unitários
- Estrutura Arrange, Act e Assert
- Cenários positivos, negativos e limites
- Regressão em sistemas
- Geração de testes com IA
- Interpretação de falhas no pytest
- Validação de regras de negócio
- Revisão crítica de código gerado por IA



Testes com IA (Unitários, E2E e API)

Testes E2E usando frameworks comuns

MÓDULO 1 SEMANA 06 - DIA 02



asCTI.SC

AGENDA

- Diferença entre testes unitários, API e E2E
- Introdução aos testes E2E
- Criação de testes E2E com frameworks comuns
- Fluxos de usuário e cenários críticos
- Introdução aos testes de API
- Testes de API com Postman
- Scripts automáticos no Postman com apoio de IA
- Exercícios práticos

OBJETIVO

Ao final da aula você será capaz de:

- Compreender o papel dos testes E2E
- Estruturar cenários baseados em fluxos de usuário
- Criar testes E2E com apoio de IA
- Compreender o papel dos testes de API
- Validar requisições e respostas no Postman
- Gerar scripts automáticos de teste com IA
- Interpretar falhas em testes E2E e API

Diferença entre testes unitários, E2E e API

Unitário

Alta velocidade e **baixa** complexidade

- Testa funções e regras isoladas
- Não abre navegador
- Não depende de sistemas externos
- Executa em milissegundos

Ex: **calcular_frete()**

API

Velocidade **intermediária** e complexidade **moderada**

- Envia requisições HTTP
- Valida endpoints e contratos
- Pode envolver autenticação e persistência
- Verifica status code, headers e resposta

Ex: **POST /login**

E2E

Execução mais **lenta** e **alta** complexidade

- Simula fluxo completo do usuário
- Executa múltiplas etapas
- Depende do sistema inteiro
- Envolve frontend, backend e ambiente completo

Ex:

Login → Carrinho → Pagamento

Introdução aos testes E2E

Testes E2E (End-to-End) validam o fluxo completo da aplicação do ponto de vista do usuário.



Os testes E2E simulam comportamento real do usuário em cenários completos.

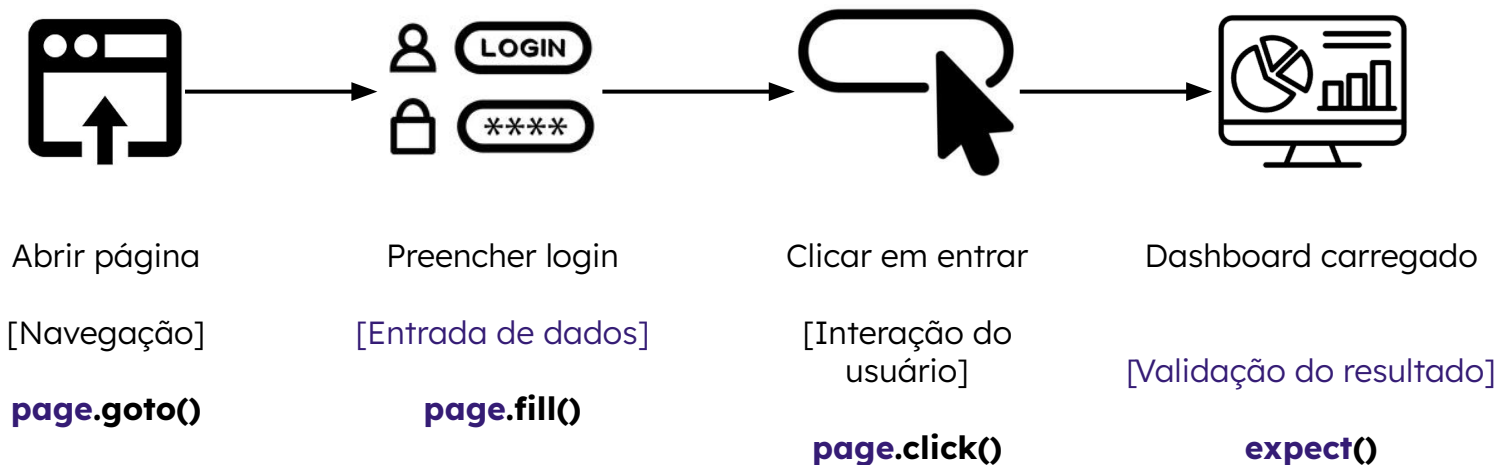
Frameworks comuns para testes E2E

Framework	Linguagem	Características
Selenium	Multi linguagem	Tradicional e amplamente utilizado
Cypress	JavaScript	Interface moderna e fácil debugging
Playwright	JavaScript / Python	Rápido, moderno e multi-browser
Puppeteer	JavaScript	Automação baseada em Chromium

```
page.goto("/login")  
page.fill("#email", "user@test.com")  
page.fill("#senha", "123456")  
page.click("#btn-login")
```

Frameworks E2E automatizam ações que normalmente seriam executadas manualmente pelo usuário.

Estrutura de um teste E2E



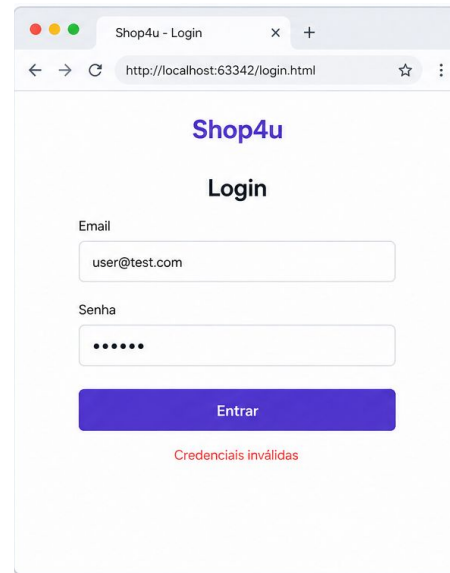
Testes E2E validam comportamento completo da aplicação a partir de interações reais

Teste E2E aplicado a um fluxo de login

Objetivo:

Automatizar o fluxo de login com Playwright, validando que credenciais válidas redirecionam o usuário para o Dashboard e que o elemento de saída fica visível após a autenticação.

Usuário acessa a **tela de login**, informa **credenciais válidas**, clica em **Entrar**, é redirecionado para o Dashboard e visualiza o **botão Sair**.



Estrutura do projeto E2E com Playwright

```
test_playwright/  
├── login.html  
├── dashboard.html  
├── test_login.py  
└── requirements.txt
```

Arquivo	Responsabilidade
login.html	Tela de autenticação
dashboard.html	Área autenticada
test_login.py	Automação E2E
requirements.txt	Dependências do projeto



A organização dos arquivos influencia diretamente a clareza e manutenção dos testes E2E automatizados.

Quando utilizar testes E2E

Quando usar E2E

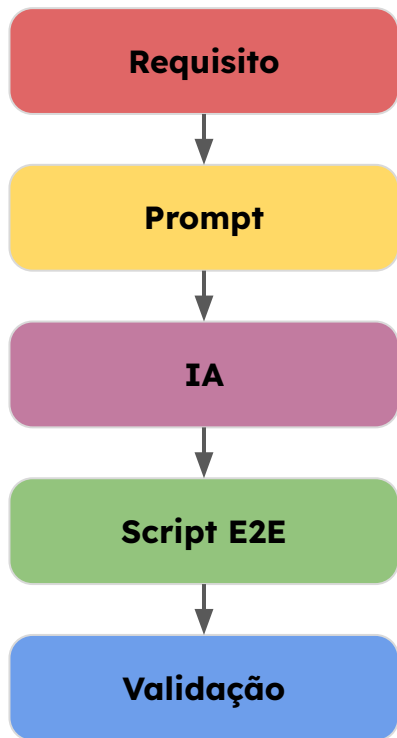
- Fluxos críticos do usuário
- Login e autenticação
- Checkout e pagamento
- Navegação entre telas
- Integração entre sistemas
- Funcionalidades de alto impacto

Quando evitar E2E

- Regras simples isoladas
- Funções matemáticas
- Pequenas validações
- Casos muito específicos
- Testes rápidos de lógica
- Regras facilmente cobertas por testes unitários

Testes E2E possuem maior custo de execução e manutenção

IA na geração de testes E2E



- 1.** Descreva o fluxo do usuário
Ex.: login → dashboard
- 2.** Informe a ferramenta
Ex.: Playwright com Python
- 3.** Defina os dados de entrada
Ex.: email e senha válidos
- 4.** Declare o comportamento esperado
Ex.: redirecionar para dashboard
- 5.** Especifique as validações
Ex.: URL correta e botão logout visível

Prompt contextualizado para testes E2E

Gere um teste E2E usando **Playwright com Python** para validar o **fluxo de login**.

Contexto:

A aplicação possui uma tela login.html com os campos:

- #email
- #senha
- #btn-login

Após login válido, o usuário deve ser **redirecionado para dashboard.html**.

Cenário:

O usuário acessa a tela de login, informa email e senha válidos, clica no botão Entrar e deve acessar o Dashboard.

Dados de entrada:

- Email: user@test.com
- Senha: 123456

Validações esperadas:

- A URL deve conter dashboard.html
- O botão "Sair" identificado por #btn-logout deve estar visível

Requisitos:

- Use pytest com Playwright em Python
- Crie um nome claro para o teste
- Use expect() do Playwright para validar URL e elementos da página
- Não gere explicações, apenas o código do teste

Teste E2E gerado pela IA

```
from pathlib import Path
from playwright.sync_api import Page, expect
```

```
def test_login_redireciona_para_dashboard(page: Page):
```

```
    base = Path(__file__).parent
    page.goto((base / "login.html").as_uri())
```

```
    page.fill("#email", "user@test.com")
    page.fill("#senha", "123456")
    page.click("#btn-login")
```

```
    expect(page).to_have_url((base / "dashboard.html").as_uri())
    expect(page.locator("#btn-logout")).to_be_visible()
```

1. Abre login.html
2. Preenche credenciais
3. Clica em Entrar
4. Valida redirecionamento
5. Confirma botão Sair

Executando o teste E2E

Instalação

```
pip install pytest-playwright  
playwright install
```

Execução

```
pytest  
pytest --headed  
pytest --headed --slowmo 700
```

Interpretação

- **Passou** → fluxo validado
- **Falhou** → comportamento divergente
- **Seletor ausente** → revisar HTML/prompt
- **URL diferente** → revisar redirecionamento

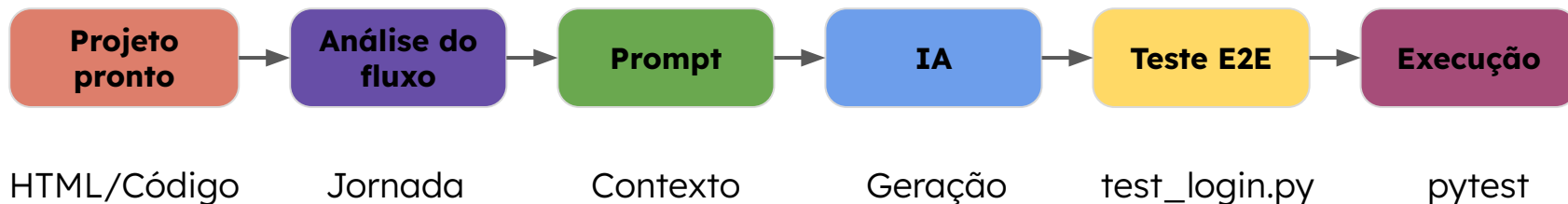
Interpretando falhas em testes E2E

Quando um teste E2E falha, a pergunta principal é: onde o fluxo quebrou?

Falha observada	Possível causa	O que revisar
Elemento não encontrado	Seletor incorreto ou elemento não carregou	HTML, seletor e tempo de carregamento
URL não mudou	Redirecionamento não ocorreu	Regra de login e evento do botão
Botão não visível	Dashboard carregou, mas o elemento esperado não apareceu	HTML do Dashboard e seletor do botão
Timeout	A condição esperada não ocorreu dentro do tempo limite	Carregamento da página, espera automática e dependências
Teste passa manualmente, mas falha automatizado	Fluxo instável	Sincronização e dependências

Exercício prático: geração de testes E2E com IA

A partir de um projeto existente, crie um prompt capaz de gerar um teste E2E executável.

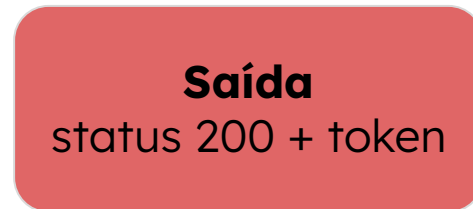
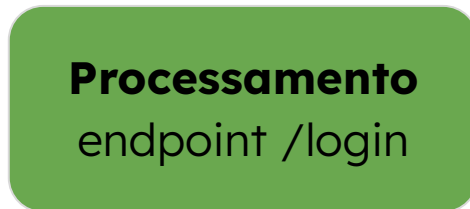
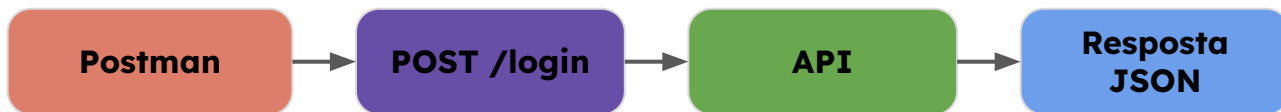


1. Fluxo do usuário
2. Ferramenta
3. Dados de entrada
4. Seletores
5. Resultado esperado
6. Validações

Introdução aos testes de API

Testes de API validam a comunicação entre sistemas por meio de requisições e respostas.

Fluxo central:



Introdução aos testes de API

Elemento	Exemplo
Método HTTP	GET, POST, PUT, DELETE
Endpoint	/login, /produtos, /pedidos
Status code	200, 201, 400, 401, 404
Corpo da resposta	JSON retornado pela API
Regras de negócio	Login válido, produto encontrado, pedido criado

Métodos HTTP e status codes

Métodos HTTP

Método	Uso comum	Exemplo
GET	Buscar dados	Buscar lista de produtos
POST	Criar ou enviar dados	Realizar login
PUT	Atualizar dados completos	Atualizar cadastro
PATCH	Atualizar parte dos dados	Alterar status do pedido
DELETE	Remover dados	Excluir item do carrinho

Métodos HTTP e status codes

Status codes

Status	Significado	Exemplo
200	Sucesso	Login realizado
201	Criado	Pedido criado
400	Requisição inválida	Campo obrigatório ausente
401	Não autorizado	Senha incorreta
404	Não encontrado	Produto inexistente
500	Erro interno	Falha no servidor

Estrutura de uma requisição e resposta

REQUISIÇÃO

POST /login

Headers:

Content-Type: application/json

Body:

```
{  
  "email": "user@test.com",  
  "senha": "123456"  
}
```



RESPOSTA

Status: 200

Body:

```
{  
  "token": "abc123",  
  "user": {  
    "email": "user@test.com"  
  }  
}
```

Testes de API com Postman

[POST ▼] http://localhost:3000/login

[Send]

BODY DA REQUISIÇÃO

```
{  
  "email": "user@test.com",  
  "senha": "123456"  
}
```

RESPOSTA DA API

```
{  
  "token": "abc123",  
  "user": {  
    "email": "user@test.com"  
  }  
}
```

Validações

- ✓ Status 200
- ✓ Token retornado
- ✓ Email do usuário retornado

Validações automáticas no Postman

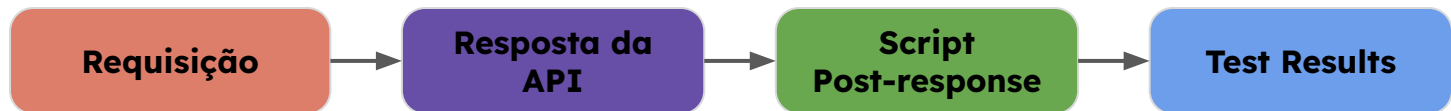
```
pm.test("Status code deve ser 200", function () {  
  pm.response.to.have.status(200);  
});
```

```
pm.test("Resposta deve conter token", function () {  
  const jsonData = pm.response.json();  
  pm.expect(jsonData).to.have.property("token");  
});
```

```
pm.test("Email do usuário deve ser retornado", function () {  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.user.email).to.eql("user@test.com");  
});
```

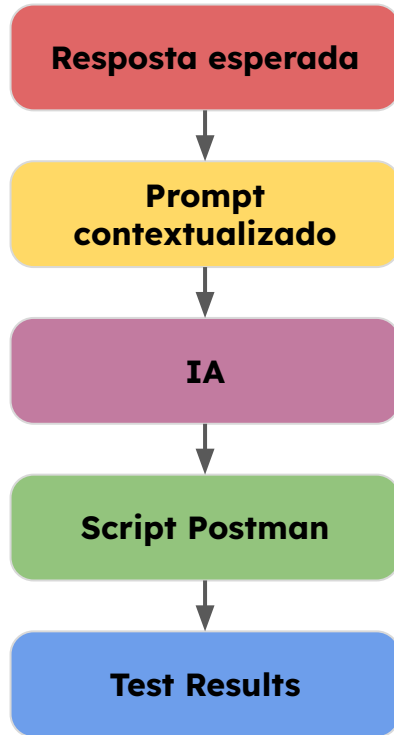
- ✓ Status code 200
- ✓ Campo token existe
- ✓ user.email está correto

Scripts automáticos no Postman



Área do Postman	Função
Body	Define os dados enviados na requisição
Headers	Informa metadados, como Content-Type
Scripts → Post-response	Executa validações após a resposta
Test Results	Mostra quais validações passaram ou falharam

IA na geração de scripts para Postman



Checklist do Prompt

- ✓ Endpoint
- ✓ Body enviado
- ✓ Status esperado
- ✓ Campos esperados na resposta
- ✓ Script para Post-response

Prompt contextualizado para scripts no Postman

Prompt:

Gere scripts de validação para o Postman usando JavaScript.

Contexto:

Estou testando o endpoint POST /login.

URL:

http://localhost:3000/login

Body enviado:

```
{  
  "email": "user@test.com",  
  "senha": "123456"  
}
```

Resposta esperada:

- Status code 200
- Campo token presente
- Campo user.email igual a "user@test.com"

Requisitos:

- Use pm.test()
- Use pm.response.to.have.status()
- Use pm.expect()
- Gere apenas o código para Scripts > Post-response

Endpoint

POST /login

Body enviado

email + senha

Resposta esperada

status 200 + token + user.email

Formato

Scripts > Post-response

Script Postman gerado pela IA

```
pm.test("Status code deve ser 200", function () {  
  pm.response.to.have.status(200);  
});
```

```
pm.test("Resposta deve conter token", function () {  
  const jsonData = pm.response.json();  
  pm.expect(jsonData).to.have.property("token");  
});
```

```
pm.test("Resposta deve conter objeto user", function () {  
  const jsonData = pm.response.json();  
  pm.expect(jsonData).to.have.property("user");  
});
```

```
pm.test("Email do usuário deve ser retornado corretamente",  
function () {  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.user.email).to.eql("user@test.com");  
});
```

No Postman:

Antes do script:

1. Abra a requisição POST /login.
2. Configure Body → raw → JSON.
3. Informe o JSON com email e senha.

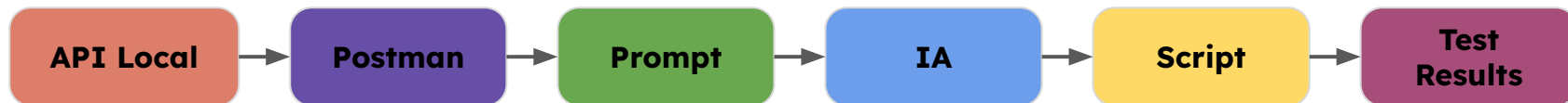
Depois:

4. Clique em Scripts.
5. Selecione Post-response.
6. Cole o script gerado.
7. Clique em Send.
8. Veja a aba Test Results.

Interpretação de falhas em testes de API

Falha observada	Possível causa	O que revisar
415 Unsupported Media Type	Body não foi enviado como JSON	Body → raw → JSON e header Content-Type: application/json
401 Unauthorized	Credenciais inválidas	Email, senha e regra de autenticação
404 Not Found	Endpoint incorreto ou inexistente	URL e rota da API
405 Method Not Allowed	Método HTTP incorreto	GET, POST, PUT, DELETE
Test Results com falha	Resposta diferente do esperado	Script, status code e campos retornados

Exercício prático: testes de API com Postman + IA



Atividade

1. Clonar o repositório da API.
2. Executar a API localmente.
3. Configurar a requisição no Postman.
4. Enviar a requisição e observar a resposta.
5. Criar um prompt contextualizado para gerar scripts.
6. Colar o script em Scripts → Post-response.
7. Executar novamente e analisar Test Results.

Checklist

- ✓ API local em execução
- ✓ Método correto
- ✓ Body em JSON
- ✓ Status esperado
- ✓ Campos obrigatórios
- ✓ Test Results analisado

Desafio: API de Recados

Cenário 1

- **POST /recados**
- **Criar recado**
- **Status 201**
- **Validar:** id, autor, mensagem, lido=false

Cenário 2

- **GET /recados**
- **Listar recados**
- **Status 200**
- **Validar:** lista e campos obrigatórios

Cenário 3

- **POST /recados**
- **Erro sem mensagem**
- **Status 400**
- **Validar:** campo error

Para cada cenário: configurar a requisição, gerar o script com IA e validar em Test Results.

Desafio: API de Recados (Cenário) - Exemplo

Gere scripts Post-response para o Postman usando JavaScript.

Endpoint:

POST `http://localhost:3000/recados`

Body enviado:

```
{  
  "autor": "Ana",  
  "mensagem": "Revisar o material da aula"  
}
```

Valide:

- status code 201
- campo id presente
- autor igual a "Ana"
- mensagem igual a "Revisar o material da aula"
- lido igual a false

Requisitos:

- Use `pm.test()`
- Use `pm.response.to.have.status()`
- Use `pm.expect()`
- Gere apenas o código, sem explicações.

Erros comuns em testes E2E e API

Tipo de teste	Erro comum	Consequência
E2E	Usar seletor incorreto	Elemento não encontrado
E2E	Validar apenas cliques, sem resultado final	Teste executa, mas não comprova o comportamento
E2E	Criar fluxos muito longos	Testes lentos e difíceis de manter
API	Usar método HTTP errado	Erro 405 Method Not Allowed
API	Enviar Body sem Content-Type JSON	Erro 415 Unsupported Media Type
API	Não validar campos da resposta	Status passa, mas contrato pode estar incorreto
IA	Aceitar teste/script gerado sem revisão	Validações incorretas ou incompletas

Principais aprendizados do dia

E2E

- Valida fluxos completos do usuário
- Usa seletores para interagir com a interface
- Deve verificar o resultado final do fluxo

API

- Valida método, endpoint, headers, body e resposta
- Verifica status code e campos retornados
- Usa scripts no Postman para automatizar validações

IA

- Ajuda a gerar testes e scripts
- Depende de prompt contextualizado
- Exige revisão crítica do desenvolvedor



Testes com IA (Unitários, E2E e API)

Análise de cobertura e riscos usando IA

MÓDULO 1 SEMANA 06 - DIA 03



SCITI.SC

Por que cobertura e risco importam?

Ter testes não significa estar bem protegido.

Cobertura

O que os testes protegem?

- ✓ Funções testadas
- ✓ Regras validadas
- ✓ Cenários cobertos

Risco

O que pode causar maior impacto?

- ✓ Login
- ✓ Pagamento
- ✓ Permissões

Testes existentes → IA analisa → Lacunas encontradas → Prioridades de teste

O que é cobertura de testes?

Cobertura de código

Linhas, funções e condições executadas

Cobertura de regras

Comportamentos esperados validados

Cobertura de cenários

Casos de sucesso, erro, limites e exceções

Cobertura alta não garante qualidade se as regras críticas não estiverem protegidas

Cobertura técnica vs. cobertura de negócio

Cobertura técnica

O teste passou por essa parte do código?

Ex: Executou um if ou uma função

Cobertura de negócio

O teste protegeu a regra esperada?

Ex: VIP tem prioridade no desconto

Cobertura de risco

O teste protegeu o que tem maior impacto se falhar?

Ex: login, pagamento, permissões

Alta cobertura técnica não garante proteção contra falhas críticas.

Exemplo: lacunas de cobertura

```
def pode_acessar(ativo, bloqueado, admin):  
    if bloqueado:  
        return False  
    if admin:  
        return True  
    return ativo
```

Testes existentes em test_pode_acessar.py:

admin=True, bloqueado=False → acessa
ativo=True, bloqueado=False → acessa

Lacunas identificadas:

- Usuário bloqueado tentando acessar
- Admin bloqueado tentando acessar
- Usuário inativo tentando acessar

A lacuna aparece ao comparar código, regra esperada e testes existentes

O que é análise de risco em testes?

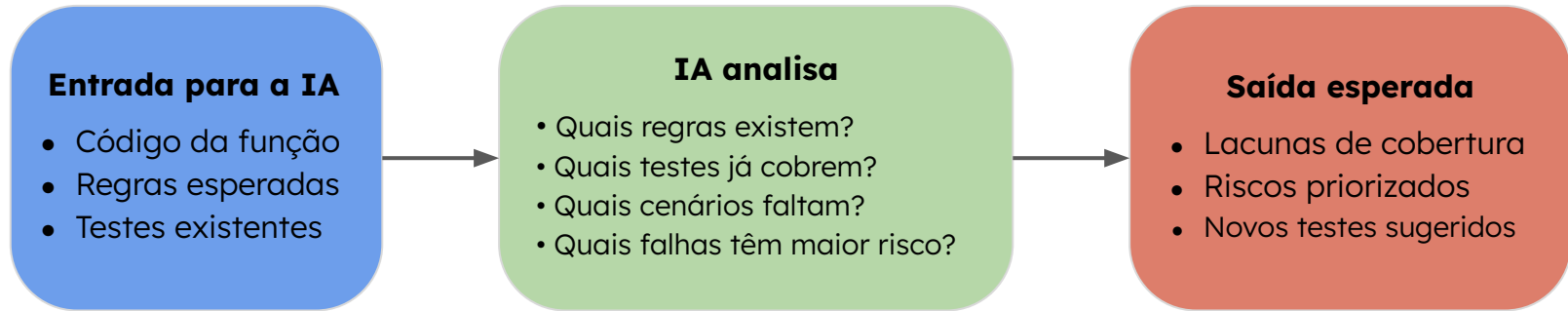
Risco indica o impacto de uma falha e ajuda a priorizar os testes.

Cenário	Risco	Por quê?
Usuário bloqueado acessa	Alto	Falha de segurança
Admin bloqueado acessa	Alto	Quebra de permissão
Usuário inativo acessa	Médio	Regra de acesso incorreta
Texto visual errado	Baixo	Baixo impacto funcional

Alto impacto + alta chance de falha → maior prioridade de teste

Como a IA ajuda na análise de cobertura e riscos

A IA pode comparar código, regras esperadas e testes existentes.



A IA acelera a revisão, mas a decisão final continua sendo técnica

Prompt para análise de cobertura e riscos

Analise o código da função, as regras esperadas e os testes existentes.

Quero que você:

1. Liste as regras de negócio.
2. Identifique quais regras estão cobertas pelos testes existentes.
3. Identifique quais regras não estão cobertas pelos testes existentes.
4. Classifique os riscos em alto, médio ou baixo e justifique.
5. Sugira novos testes para reduzir os principais riscos.

Considere:

- cenários positivos;
- cenários negativos;
- casos de borda;
- entradas inválidas;
- prioridade entre regras.

Saída esperada:

- ✓ Regras identificadas
- ✓ Cobertura atual
- ✓ Lacunas encontradas
- ✓ Riscos priorizados
- ✓ Novos testes sugeridos

Conclusão - Testes com IA (Unitários, E2E e API)

Dia 1 — Testes unitários assistidos por IA

- Fundamentos de testes unitários e estrutura Arrange / Act / Assert
- Geração de testes com IA a partir de regras de negócio
- Cenários positivos, negativos, entradas inválidas e edge cases

Dia 2 — Testes E2E e testes de API com IA

- Automação de fluxos completos com Playwright
- Testes de API com Postman, requisições, respostas e status codes
- Scripts automáticos no Postman com apoio de IA

Dia 3 — Cobertura, riscos e mini projeto

- Análise de cobertura técnica, de negócio e de risco
- Identificação de lacunas e priorização de testes com IA
- Aplicação dos conceitos no mini projeto e revisão dos resultados



SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_



SCITI.SC

Correção guiada dos exercícios

Exercício	Ponto de atenção	O que revisar
Teste E2E	Fluxo completo do usuário	Seletores, redirecionamento e validação final
Teste de API	Configuração da requisição	Método, URL, headers e body
Script Postman	Validações automáticas	Status code, campos obrigatórios e valores esperados
Uso de IA	Qualidade do código gerado	Se o script/teste está coerente com o cenário

O teste valida comportamento?

A IA inventou algo?

A falha foi bem interpretada?

O prompt precisa melhorar?

Desafio 1: validação de senha

```
def validar_senha(senha):
```

```
    if len(senha) < 8:
```

```
        return False
```

```
    if senha.islower():
```

```
        return False
```

```
    if senha.isalpha():
```

```
        return False
```

```
    return True
```

Objetivo da atividade

Gerar testes unitários usando pytest para validar diferentes tipos de senha.

Requisitos

- Utilizar IA para auxiliar na geração dos testes;
- Criar pelo menos 4 cenários;
- Incluir pelo menos 1 edge case;
- Executar os testes com **pytest**;
- Interpretar os resultados obtidos.

1. Quais regras tornam a senha inválida?
2. Quais cenários precisam ser protegidos pelos testes?

Desafio 2: cálculo de desconto

```
def calcular_desconto(valor_compra, cliente_vip):
```

```
    if valor_compra < 0:  
        raise ValueError("Valor inválido")
```

```
    if cliente_vip:  
        return valor_compra * 0.85
```

```
    if valor_compra >= 300:  
        return valor_compra * 0.90
```

```
    return valor_compra
```

1. Quais regras definem os descontos aplicados?
2. Existe prioridade entre as regras?
3. Quais cenários devem ser validados pelos testes?

Objetivo da atividade

Validar regras de desconto e prioridade de execução através de testes automatizados.

Requisitos

- Gerar testes com apoio de IA
- Criar cenários positivos e negativos
- Validar tratamento de exceção
- Identificar regras prioritárias
- Executar os testes usando pytest
- Interpretar os resultados gerados

Desafio 3: classificação de chamados

```
def classificar_chamado(tipo_problema, cliente_premium):
```

```
    if tipo_problema == "sem_sinal":  
        return "alta"
```

```
    if tipo_problema == "cobranca" and cliente_premium:  
        return "alta"
```

```
    if tipo_problema == "cobranca":  
        return "media"
```

```
    return "baixa"
```

1. Quais condições geram prioridade alta?
2. Quando a prioridade deve ser média?
3. Qual cenário retorna prioridade baixa?
4. A ordem das regras influencia o resultado?

Objetivo da atividade

Validar fluxos condicionais e regras de prioridade na classificação de chamados.

Requisitos

- Gerar testes com apoio de IA
- Validar diferentes tipos de problema
- Testar clientes comuns e premium
- Identificar regras prioritárias
- Executar os testes usando pytest
- Interpretar os resultados gerados